

Competitive Programming

Section 44 Editorial: String Algorithms II

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Trie Prefix Count	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
2.7	Tests that matter	6
3	B. Palindrome Queries	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
3.7	Tests that matter	8
4	C. Distinct Substrings	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
4.7	Tests that matter	10
5	What to carry forward	11
6	Testing discussion	12

1 Editorial overview

1.1 Problem-specific proof map

A. Trie Prefix Count. Incrementing every trie node visited by an inserted word makes that node's count equal to the number of words sharing its prefix. Following a query prefix therefore returns the desired count in its length.

B. Palindrome Queries. The local constraints permit direct mirrored comparison: a substring is a palindrome exactly when every symmetric pair is equal. This is $O(\text{length})$ per query and avoids probabilistic hash collisions.

C. Distinct Substrings. The local task uses a suffix automaton. State v contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ distinct substring lengths, so summing these over non-root states is linear in the number of automaton states. Enumeration of all substrings is only a small-input stress-testing oracle.

1.2 Detailed solution guide

A. Trie Prefix Count. Each trie node represents exactly one prefix: the string on the path from the root to that node. During insertion, increment `cnt` at the root and after moving to each next node. Therefore, after all insertions, `cnt[v]` is the number of dictionary words whose path passes through v , which is precisely the number of words with that prefix. A query walks the prefix characters; a missing edge means zero matches.

B. Palindrome Queries. For the local constraints, comparing mirrored characters is the clearest exact solution. For substring $[l, r]$, compare $s[l]$ with $s[r]$, then $s[l+1]$ with $s[r-1]$, and so on. If all pairs match, the substring equals its reverse. If one pair differs, the two strings differ at a mirrored position, so it is not a palindrome. In a larger contest version, the same condition can be checked with forward and reverse rolling hashes.

C. Distinct Substrings. The local solution is a suffix automaton. Each state contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ new substrings: these are exactly the lengths whose end-position class first appears at that state. Summing over non-root states gives the answer. Enumerating every substring into a set is correct only for tiny random tests used to validate the automaton.

Common pitfalls. Trie counters count prefixes, not terminal words only. Palindrome queries must convert input indices consistently. For distinct substrings, enumeration is pedagogical and exact at small limits, but a suffix automaton is required for large external tasks.

This section shows three string representations.

- A uses a trie. Each inserted word increments counters on the nodes along its path, so the counter at the prefix endpoint is exactly the number of words with that prefix.
- B checks palindromes by comparing mirrored characters. For larger limits this can be replaced by forward/reverse hashes, but the local proof is exact.

- C counts distinct substrings with a suffix automaton. The local invariant is the contribution `len[v] - len[link[v]]` per non-root state; enumeration is reserved for tiny brute-force tests.

The editorial code uses the same asymptotically appropriate representation as the local contract; brute force appears only as a testing oracle.

2 A. Trie Prefix Count

2.1 Restatement

Given dictionary words and prefix queries, count words having each prefix.

2.2 Observations and algorithm

Each trie node is one prefix. Increment its counter for every inserted word visiting it; walk a query to its endpoint and return that counter, or zero on a missing edge. Inserting **apple**, **ape**, **bat** makes query **ap** return two.

2.3 Correctness proof

After every insertion, a node counter equals the words with that node's prefix, so the query endpoint is exactly the requested count. QED.

2.4 Complexity

Work is linear in total inserted and queried characters; memory is one node per distinct prefix.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Node{array<int,26> nx{}; int cnt=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; v
```

2.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().splitlines(); n,q=map(int,lines[0].split()); nxt=[]; cnt=[]
    nxt.append({}); cnt.append(0)
    for w in lines[1:1+n]:
        v=0; cnt[v]+=1
        for ch in w:
```

```

        if ch not in nxt[v]: nxt[v][ch]=len(nxt); nxt.append({}); cnt.append(0)
        v=nxt[v][ch]; cnt[v]+=1
    out=[]
    for p in lines[1+n:1+n+q]:
        v=0; ok=True
        for ch in p:
            if ch not in nxt[v]: ok=False; break
            v=nxt[v][ch]
        out.append(str(cnt[v] if ok else 0))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.7 Tests that matter

Test duplicate words, one word that prefixes another, an absent prefix, and a one-character query.

3 B. Palindrome Queries

3.1 Restatement

For each substring query, print whether the substring is a palindrome.

3.2 Observations and algorithm

The local bounds permit exact mirrored comparison. Convert inclusive 1-based $[l, r]$, compare outward characters, and stop on a mismatch. In `abacaba`, $[2, 4]$ is `bac` (NO) while $[3, 5]$ is `aca` (YES).

3.3 Correctness proof

A substring equals its reverse iff each mirrored character pair is equal, which is exactly the two-pointer condition. QED.

3.4 Complexity

Each query is $O(r-l+1)$ time and $O(1)$ extra memory; this exact approach is only suitable for the local small-query contract.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(c
```

3.6 Full Python solution

```
import sys
def main():
    s=sys.stdin.readline().strip(); q=int(sys.stdin.readline()); out=[]
    for _ in range(q):
        l,r=map(int,sys.stdin.readline().split()); sub=s[l-1:r]; out.append("YES" if sub==su
    print("\n".join(out))
if __name__=="__main__": main()
```

3.7 Tests that matter

Test singleton, full string, even and odd lengths, and a mismatch in the outer pair.

4 C. Distinct Substrings

4.1 Restatement

Print the number of distinct non-empty substrings of a string.

4.2 Observations and algorithm

Build a suffix automaton. State v contributes $\text{len}[v] - \text{len}[\text{link}[v]]$ new substring lengths; sum this over non-root states. For `ababa`, the answer is 9.

4.3 Correctness proof

Suffix links partition represented lengths into disjoint intervals, so every distinct nonempty substring belongs to exactly one state's contribution. Clones restore this invariant when a transition needs a shorter maximum length. QED.

4.4 Complexity

There are at most $2n-1$ states. The fixed-alphabet version is $O(n)$; the map-based listing is $O(n \log \text{alphabet})$.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct St{map<char,int> nx; int link=-1,len=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(cin>>s)) return 0; vec
```

4.6 Full Python solution

```
import sys
def main():
    s=sys.stdin.readline().strip(); next=[]; link=[]; length=[]
    next.append({}); link.append(-1); length.append(0); last=0
    for ch in s:
        cur=len(next); next.append({}); length.append(length[last]+1); link.append(0); p=las
```

```

while p!=-1 and ch not in next[p]: next[p][ch]=cur; p=link[p]
if p==-1: link[cur]=0
else:
    q=next[p][ch]
    if length[p]+1==length[q]: link[cur]=q
    else:
        clone=len(next); next.append(next[q].copy()); length.append(length[p]+1); li
        while p!=-1 and next[p].get(ch)==q: next[p][ch]=clone; p=link[p]
        link[q]=link[cur]=clone
    last=cur
print(sum(length[v]-length[link[v]] for v in range(1,len(next))))
if __name__=="__main__": main()

```

4.7 Tests that matter

Test a, aaa, all distinct characters, and an alternating string.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

6 Testing discussion

Brute-force small trie prefixes by `word.startswith(prefix)`, palindrome queries by reverse comparison, and distinct substrings with a set of all slices. Then run:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```